Task1_1

This code works by fetching an image of an insect from the drive, and outputs the image. The code also outputs the data type of the image, image format, colour mode, and size of the image. Imshow outputs the image, and plt shows the image.

```
<class 'PIL.PngImagePlugin.PngImageFile'>
PNG
RGB
(464, 464)
```

```
#loading an image
import matplotlib.pyplot as plt
# example of loading an image with the Keras API
from keras.preprocessing.image import load_img
# load the image
img_path ='/content/drive/MyDrive/Colab Notebooks/Assets/1_Gammar 2021_1_2021_06_03-13-19-23-856.PNG'
img = load_img(img_path)
# report details about the image
print(type(img))
print(img.format)
print(img.mode)
print(img.size)
# show the image using matplotlib
plt.imshow(img)
plt.axis('off') # Turn off axis labels
plt.show()
```

```
<class 'PIL.PngImagePlugin.PngImageFile'>
PNG
RGB
(464, 464)
```

Task1_2

In this task, keras is imported to load an image, converts it to a numpy array, and convert it back it an image. By using the path of the image in drive, it stores the image as PIL image object and then prints the data type of the numpy array values. It also prints the dimensions of the image.

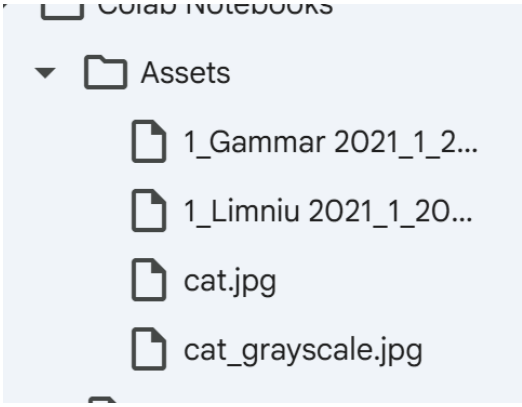
```
•••  <class 'PIL.PngImagePlugin.PngImageFile'>
     float32
     (476, 476, 3)
     <class 'PIL.PngImagePlugin.PngImageFile'>
```

```python
# example of converting an image with the Keras API
from keras.preprocessing.image import load_img
from keras.preprocessing.image import img_to_array
from keras.preprocessing.image import array_to_img
# load the image
img_path ='/content/drive/MyDrive/Colab Notebooks/Assets/1_Limniu 2021_1_2021_06_02-13-08-37-941.PNG'
img = load_img(img_path)
print(type(img))
# convert to numpy array
img_array = img_to_array(img)
print(img_array.dtype)
print(img_array.shape)
# convert back to image
img_pil = array_to_img(img_array)
print(type(img))
```

Task1_3

This task takes an image of a cat from the assets folder in the drive through using its path to convert the image in to black and white (greyscale). I gave a path to save the new converted image and a file name. After saving, it prints out details of the image like the format, mode and size and outputs the new image using matplotlib.

```
<class 'PIL.Image.Image'>
None
RGB
(1920, 2879)
```



It stores the new image in the specified path and the file gets named as stated in the code.

```python
import matplotlib.pyplot as plt
import os
# example of saving an image with the Keras API
from keras.preprocessing.image import load_img
from keras.preprocessing.image import save_img
from keras.preprocessing.image import img_to_array
# load image as as grayscale
img_path = '/content/drive/MyDrive/Colab Notebooks/Assets/cat.jpg'
img = load_img(img_path, color_mode='grayscale')
# convert image to a numpy array
img_array = img_to_array(img)
# save the image with a new filename
save_path_dir = '/content/drive/MyDrive/Colab Notebooks/Assets/'
save_filename = 'cat_grayscale.jpg'
full_save_path = os.path.join(save_path_dir, save_filename)
save_img(full_save_path, img_array)
# load the image to confirm it was saved correctly
img = load_img(full_save_path)
print(type(img))
print(img.format)
print(img.mode)
print(img.size)
# show the image using matplotlib
plt.imshow(img)
plt.axis('off') # Turn off axis labels
plt.show()
```

Task1_4

This task loads MNIST dataset, which is big collection of handwritten digit images from 0 to 9. It uses Keras and splits it into training set and a test set. It then prints the shape of both sets which tells you how many images there are and their dimensions in pixels. The last two outputs are basic statistics about the pixel values such as the minimum, maximum, mean and standard deviation. This basically checks that both sets of data are loaded correctly and to understand the properties of the image data.

```
•••  Downloading data from https://storage.googleapis.com/tensorflow/tf-
     11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
     Train (60000, 28, 28) (60000,)
     Test ((10000, 28, 28), (10000,))
     Train 0 255 33.318421449829934 78.56748998339798
     Train 0 255 33.791224489795916 79.17246322228644
```

```python
# load and summarize the MNIST dataset
from keras.datasets import mnist
# load dataset
(train_images, train_labels), (test_images,
test_labels) = mnist.load_data()
# summarize dataset shape
print('Train', train_images.shape,
train_labels.shape)
print('Test', (test_images.shape,
test_labels.shape))
# summarize pixel values
print('Train', train_images.min(),
train_images.max(), train_images.mean(),
train_images.std())
print('Train', test_images.min(),
test_images.max(), test_images.mean(),
test_images.std())
```

Task1_5

This code normalizes the image data so that the pixel values are small numbers between 0 and 1 by rescaling, which makes it much easier and faster for the program to process the data. The output shows that at the start the image had pixel values of minimum of 0 and maximum of 255 while at the end, it got normalized to a minimum of 0 and maximum of 1.

```
•••  Train min=0.000, max=255.000
     Test min=0.000, max=255.000
     Batches train=938, test=157
     Batch shape=(64, 28, 28, 1), min=0.000, max=1.000
```

```python
# example of normalizing an image dataset (MNIST)
from keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# load dataset
(trainX, trainY), (testX, testY) = mnist.load_data()

# reshape dataset to have a single channel
width, height, channels = trainX.shape[1], trainX.shape[2], 1
trainX = trainX.reshape((trainX.shape[0], width, height, channels))
testX = testX.reshape((testX.shape[0], width, height, channels))

# confirm scale of pixels
print('Train min=%.3f, max=%.3f' % (trainX.min(), trainX.max()))
print('Test min=%.3f, max=%.3f' % (testX.min(), testX.max()))

# create generator (1.0/255.0 = 0.003921568627451)
datagen = ImageDataGenerator(rescale=1.0/255.0)

# Note: there is no need to fit the generator in this case
# prepare a iterators to scale images

train_iterator = datagen.flow(trainX, trainY, batch_size=64)
test_iterator = datagen.flow(testX, testY, batch_size=64)

print('Batches train=%d, test=%d' % (len(train_iterator),
len(test_iterator)))

# confirm the scaling works
batchX, batchy = next(train_iterator)
print('Batch shape=%s, min=%.3f, max=%.3f' % (batchX.shape,
batchX.min(), batchX.max()))
```

Task1_6

This task centres the pixel values, which means the mean of the pixel values is subtracted from each pixel so that the values are centred around 0 instead of their original range. It first reshapes the images to add a single channel dimension, then prints the average pixel value for both the training and test sets before any centring is done. The featurewise_center = true creates a generator to calculate the main and subtract it from the images. It lastly, prints the mean of a single batch and then the entire training dataset, both of which come out close to 0.

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Means train=33.318, test=33.791
Data Generator Mean: 33.318
(64, 28, 28, 1) -0.6572954
(60000, 28, 28, 1) -1.9512918e-05
```

```python
# example of centering a image dataset
from keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# load dataset

(trainX, trainy), (testX, testy) = mnist.load_data()

# reshape dataset to have a single channel

width, height, channels = trainX.shape[1], trainX.shape[2], 1
trainX = trainX.reshape((trainX.shape[0], width, height,
channels))
testX = testX.reshape((testX.shape[0], width, height,
channels))
# report per-image mean
print('Means train=%.3f, test=%.3f' % (trainX.mean(),
testX.mean()))

# create generator that centers pixel values
datagen = ImageDataGenerator(featurewise_center=True)
# calculate the mean on the training dataset
datagen.fit(trainX)

print('Data Generator Mean: %.3f' % datagen.mean.item())
# demonstrate effect on a single batch of samples
iterator = datagen.flow(trainX, trainy, batch_size=64)
# get a batch
batchX, batchy = next(iterator)
# mean pixel value in the batch
print(batchX.shape, batchX.mean())
# demonstrate effect on entire training dataset
iterator = datagen.flow(trainX, trainy,
batch_size=len(trainX), shuffle=False)
# get a batch
batchX, batchy = next(iterator)
# mean pixel value in the batch
print(batchX.shape, batchX.mean())
```

Task1_7

This tasks basically standardises an image dataset. Similarly, as task1_6, it first subtract the mean to centre the values around 0 and then divides it by the standard deviation to make the spread of values consistent. It loads and reshapes the MNIST dataset the same way as before, then prints both the mean and standard deviation of the training and test sets before any changes are made. It prints the mean and standard deviation and the mean close to 0 and standard deviation close to 1.

```
Statistics train=33.318 (78.567), test=33.791 (79.172)
Data Generator mean=33.318, std=78.567
(64, 28, 28, 1) -0.0014777204 0.9991395
(60000, 28, 28, 1) -3.4560264e-07 0.9999998
```

```python
# example of standardizing a image dataset
from keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# load dataset
(trainX, trainy), (testX, testy) = mnist.load_data()
# reshape dataset to have a single channel
width, height, channels = trainX.shape[1], trainX.shape[2], 1
trainX = trainX.reshape((trainX.shape[0], width, height, channels))
testX = testX.reshape((testX.shape[0], width, height, channels))
# report pixel means and standard deviations
print('Statistics train=%.3f (%.3f), test=%.3f (%.3f)' % (trainX.mean(), trainX.std(),
testX.mean(), testX.std()))
# create generator that centers pixel values
datagen = ImageDataGenerator(featurewise_center=True,
featurewise_std_normalization=True)
# calculate the mean on the training dataset
datagen.fit(trainX)
print('Data Generator mean=%.3f, std=%.3f' % (datagen.mean.item(),
datagen.std.item()))
# demonstrate effect on a single batch of samples
iterator = datagen.flow(trainX, trainy, batch_size=64)
# get a batch
batchX, batchy = next(iterator)
# pixel stats in the batch
print(batchX.shape, batchX.mean(), batchX.std())
# demonstrate effect on entire training dataset
iterator = datagen.flow(trainX, trainy, batch_size=len(trainX), shuffle=False)
# get a batch
batchX, batchy = next(iterator)
# pixel stats in the batch
print(batchX.shape, batchX.mean(), batchX.std())
```